SQL Project: Bookstore Management System

🛠️ Tools Used

- *PostgreSQL – Database Management*
- *PgAdmin – Query Execution & Database Handling*
- *Microsoft Excel / CSV Files – Data Import*
- *Canva – Project Presentation Design*
- *SQL – Data Querying & Analysis*

Database Schema (Books, Customers, Orders)

BOOKS
- Book_ID (PK)
- Title
- Author
- Genre
- Published_Year
- Price
- Stock

CUSTOMERS
- Customer_ID (PK)
- Name
- Email
- Phone
- City
- Country

ORDERS
- Order_ID (PK)
- Customer_ID (FK)
- Book_ID (FK)
- Order_Date
- Quantity
- Total_Amount

Creating & importing Books Table

```sql
--Creating Books Table
Create table books(
Book_ID serial primary key,
Title varchar(100),
Author varchar(100),
Genre varchar(50),
Published_Year int,
Price Numeric(10,2),
Stock int
);

--Import data into table
COPY books(Book_ID, Title, Author, Genre, Published_Year, Price, Stock)
FROM 'D:\POSTERES_SQL\ST - SQL ALL PRACTICE FILES-2\Project_PgSql\Books.csv'
CSV HEADER;
```

Creating & importing Customer Table

```sql
--Creating Customer Table
Create table Customers(
Customer_ID serial primary key,
Name Varchar(100),
Email Varchar(100),
Phone Varchar(15),
City Varchar(50),
Country Varchar(150)
);
--Import data into table
COPY Customers(Customer_ID, Name, Email, Phone, City, Country)
FROM 'D:\POSTERES_SQL\ST - SQL ALL PRACTICE FILES-2\Project_PgSql\Customers.csv'
CSV HEADER;
SELECT * FROM CUSTOMERS;
```

Creating & importing Orders Table

```
--Create Orders Table
Create table Orders(
Order_ID serial primary key,
Customer_ID int references Customers(Customer_ID),
Book_ID int references books(Book_ID),
Order_Date  Date,
Quantity int,
Total_Amount numeric(10,2)
);
--Import data into table
COPY Orders(Order_ID, Customer_ID, Book_ID, Order_Date, Quantity, Total_Amount)
FROM 'D:\POSTERES_SQL\ST - SQL ALL PRACTICE FILES-2\Project_PgSql\Orders.csv'
CSV HEADER;
select * from Orders;
```

--Questions--

--Retrive all the books in the "Fiction" Genre:

```sql
SELECT
    *
FROM
    BOOKS
WHERE
    GENRE = 'Fiction';
```

	book_id [PK] integer	title character varying (100)	author character varying (100)	genre character varying (50)	published_year integer	price numeric (10,2)
1	4	Customizable 24hour product	Christopher Andrews	Fiction	2020	43.52
2	22	Multi-layered optimizing migration	Wesley Escobar	Fiction	1908	39.23
3	28	Expanded analyzing portal	Lisa Coffey	Fiction	1941	37.51
4	29	Quality-focused multi-tasking challenge	Katrina Underwood	Fiction	1905	31.12
5	31	Implemented encompassing conglomerat...	Melissa Taylor	Fiction	2010	21.23

--Questions--

--Retrive the total stock of books available:

```
SELECT
    SUM(STOCK) AS TOTAL_STOCKS
FROM
    BOOKS;
```

	total_stocks bigint
1	25056

--Calculate the total revenue generated from all orders:

```sql
SELECT
    SUM(TOTAL_AMOUNT) AS REVENUE
FROM
    ORDERS;
```

revenue numeric
75628.66

--Retrive the total numbers of books sold for each genre:

```sql
SELECT
    B.GENRE,
    SUM(O.QUANTITY) AS TOTAL_BOOKS_SOLD
FROM
    ORDERS O
    JOIN BOOKS B
    ON O.BOOK_ID = B.BOOK_ID
GROUP BY
    B.GENRE;
```

#OUTPUT#

	genre character varying (50)	total_books_sold bigint
1	Romance	439
2	Biography	285
3	Mystery	504
4	Fantasy	446
5	Fiction	225
6	Non-Fiction	351
7	Science Fiction	447

--Find the avrage price of book in fantacsy genre:

```sql
SELECT
    AVG(PRICE) AS AVRAGE_PRICE
FROM
    BOOKS
WHERE
    GENRE = 'Fantasy';
```

	avrage_price numeric
1	25.9816901408450704

--List the customers who have palced atleast 2 orders:

```
SELECT
    O.CUSTOMER_ID,
    C.NAME,
    COUNT(O.ORDER_ID) AS ORDER_COUNT
FROM
    ORDERS O
    JOIN CUSTOMERS C ON O.CUSTOMER_ID = C.CUSTOMER_ID
GROUP BY
    O.CUSTOMER_ID,
    C.NAME
HAVING
    COUNT(ORDER_ID) >= 2;
```

customer_id integer	name character varying (100)	order_count bigint
225	Christopher Mccullough	2
418	Kiara Blankenship MD	3
322	William Cameron	3
325	Emily Vargas	4
376	Justin Donaldson	2
486	Melanie Kelly	2
461	Crystal Pierce	3
2	Crystal Clements	2
149	Jason Robinson	3
173	Victoria Dixon	2

--Calculate the stock remaining after fulfilling all orders:

```sql
SELECT
    B.BOOK_ID,
    B.TITLE,
    B.STOCK,
    COALESCE(SUM(O.QUANTITY), 0) AS ORDER_QUANTITY,
    B.STOCK - COALESCE(SUM(O.QUANTITY), 0) AS REMAINING_QUANTITY
FROM
    BOOKS B
    LEFT JOIN ORDERS O ON B.BOOK_ID = O.BOOK_ID
GROUP BY
    B.BOOK_ID
ORDER BY
    B.BOOK_ID;
```

	book_id [PK] integer	title character varying (100)	stock integer	order_quantity bigint	remaining_quantity bigint
1	1	Configurable modular throughput	100	3	97
2	2	Persevering reciprocal knowledge user	19	0	19
3	3	Streamlined coherent initiative	27	5	22
4	4	Customizable 24hour product	8	0	8
5	5	Adaptive 5thgeneration encoding	16	8	8

Thank You